UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SOFTWARE ENGINEERING

Software Design: Design Patterns

THE WATERFALL SOFTWARE PROCESS MODEL

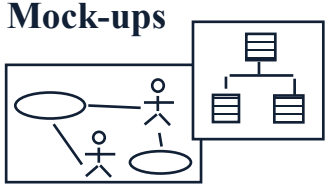
Requirements Engineering → **System Design** → **Software and UI Design** → **Implementation** → **Testing** → **Operation and Maintenance**

Requirements collected via:

- Interviews with Stakeholders
- Personas
- Stories and Scenarios

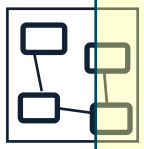
Specified using:

- Use Cases
- Natural Language
- Domain Models
- Mock-ups



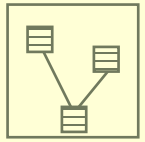
Define System Architecture

- Requirements are allocated to software sub-systems
- Sub-systems are allocated to hardware resources
- Architectural Patterns



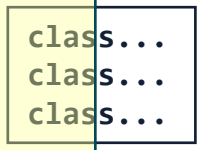
Define Subsystems

- Objects required to realize each subsystem are defined.
- Software Design Patterns
- Usability Engineering
- High-fidelity Wireframing



Each Subsystem is implemented

- Source code and other artifacts
- Clean Code
- Frameworks and ORMs
- Focus on Software Quality



Ensure the Software satisfies customers

- Code inspections
- Functional Testing (unit, integration, system testing)
- Usability Testing

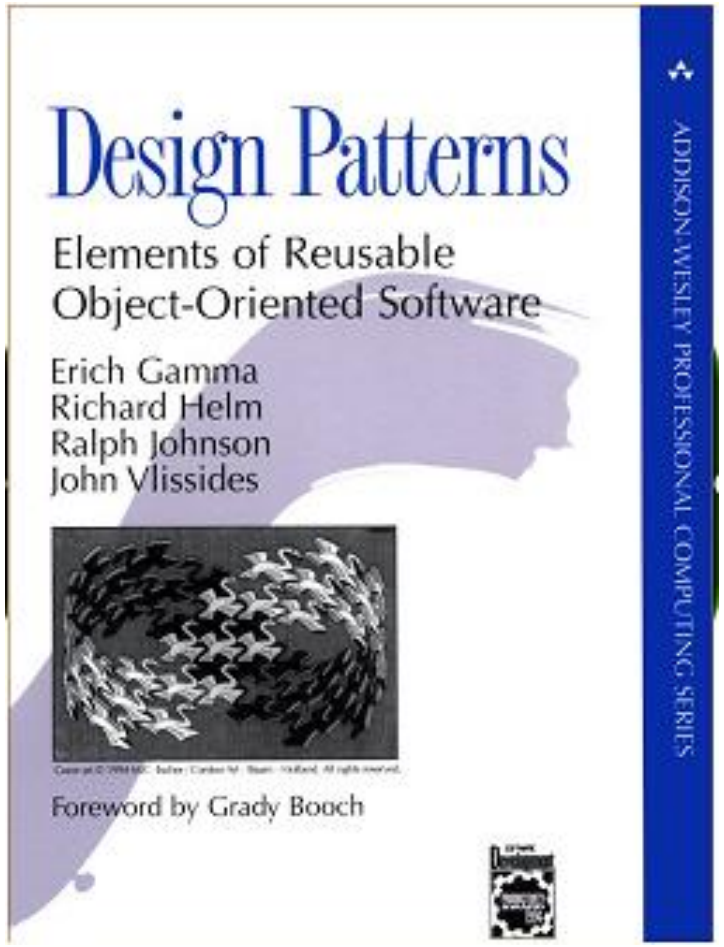
System is put into practical use

Maintance will be required at some point

- To fix errors that were not discovered in previous phases
- To adapt the software to changes in requirements on in its environment

What is a Design Pattern?

- A design pattern
  - Is a common solution to a recurring problem in design
  - Abstracts a recurring design structure
  - Comprises class and/or object
    - Dependencies
    - Structures
    - Interactions
    - Conventions
  - Names & specifies the design structure explicitly
  - Distils design experience



History of Design Patterns

- Architect Christopher Alexander
  - *A Pattern Language: Towns, Buildings, Construction* (1977)
- “Gang of four”
  - Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides
  - *Design Patterns: Elements of Reusable Object-Oriented Software (1995)*
    - *Steven John Metsker, Design Patterns in Java, Pearson, 2003*
- Many since
  - Conferences, symposia, books
- It is not unusual that single developers proposes new patterns and disseminate them informally via blogs, magazines or other sources

What Is a Design Pattern?

- A design pattern has 4 basic parts:
  - 1. Name
  - 2. Problem
  - 3. Solution
  - 4. Consequences and trade-offs of application
- Language- and implementation-independent
- A “micro-architecture”
- No mechanical application
  - The solution needs to be translated into concrete terms in the application context by the developer

Goals

- Codify good design
  - Distil and disseminate experience
  - Aid to novices and experts alike
  - Abstract how to think about design
- Give design structures explicit names
  - Common vocabulary
  - Reduced complexity
  - Greater expressiveness
- Capture and preserve design information
  - Articulate design decisions succinctly
  - Improve documentation
- Facilitate restructuring/refactoring
  - Patterns are interrelated
  - Additional flexibility

Gang of Four (GoF) patterns

Creational

Design Patterns	Description
Builder	Separates the construction of a complex object from its representation
Abstract Factory	Provides an interface for creating families of related objects without specifying their concrete classes
Factory Method	Defines an interface to create an object but lets subclasses decide which class to instantiate
Prototype	Specify the type of object to be created using a prototype instance, and create new objects by copying this prototype
Singleton	Ensures that the class has only one instance and provides a way of access

Singleton

DP: Singleton (Creational)

- **Name**: Singleton

- **Purpose** : Build classes that support one and only one object instance and provide a global access point to that instance

Example

```
class SingletonClass {
   private static SingletonClass singletonObject=null;

private SingletonClass() {
   // Optional Code
}

public static SingletonClass getSingletonObject() {
   if (singletonObject == null) {
          singletonObject = new SingletonClass(); }
   return singletonObject;
}

public Object clone()
throws CloneNotSupportedException { throw new
   CloneNotSupportedException();
}
}
```

- The singletonObject reference is static, so it persists as objects change
- The singletonObject reference is private, so it can't be changed from outside the class
- The constructor is private so that it can only be invoked by getSingletonObject()
- The reference to SingletonClass can be obtained by anyone by calling the static public method getSingletonObject(), which could instantiate the object if it did not already exist
- The ability to copy the object with the clone override is disabled, which throws an exception

Consequences

- At most one object of the singleton class can be instantiated
  - In some variants of the Singleton object it is possible to have a fixed number of instances (for example, at most 3) declared as an array of references and modified accordingly by the getSingletonObject
- Singleton pattern is better than a static object
  - Because the creation of the object can be customized
  - Because cloning can be disabled

  - We used that solution for DatabaseConnection object

Builder

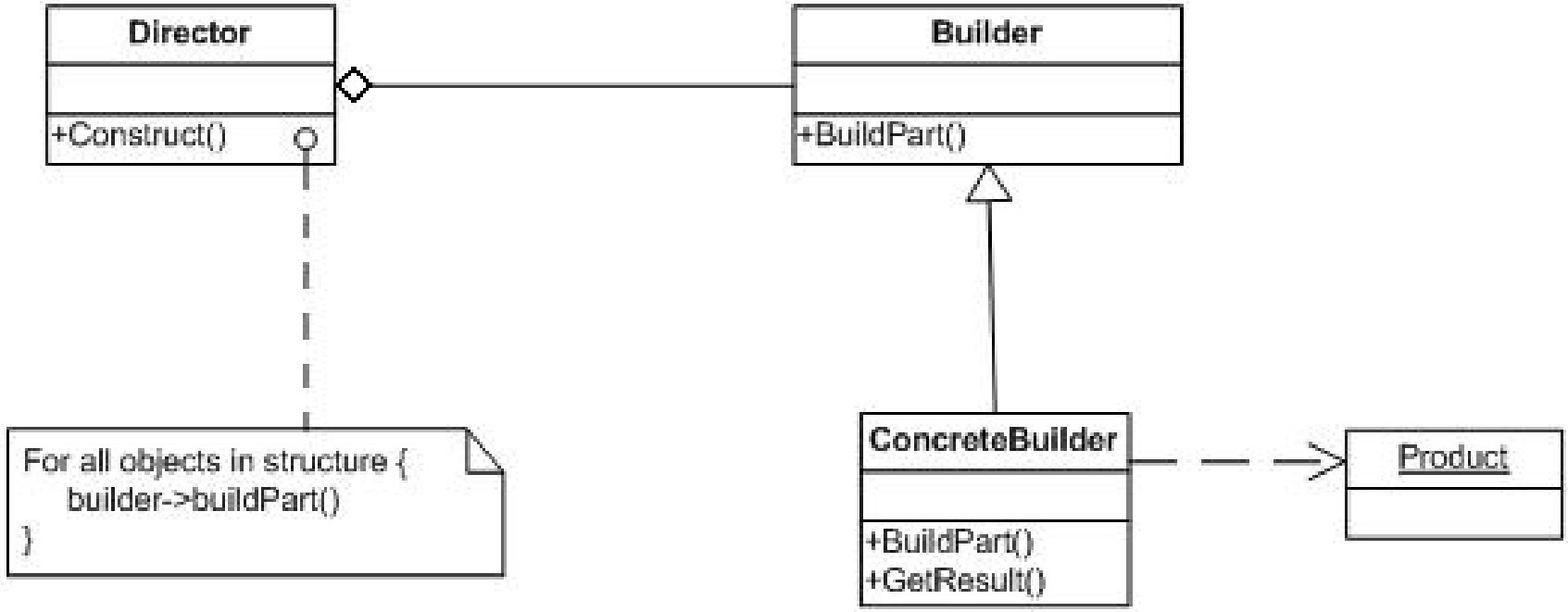
Builder

- There is a whole family of patterns (Builder and Factory) that all serve to address the problem of complex classes for which the basic object-oriented mechanism of constructors is not sufficient to guarantee sufficient flexibility and efficiency

- **Name** : Builder
- **Purpose** : It makes the construction of an object flexible and decomposable, separating it from the actual constructor of the object

Solution

- The single object Product is built by ConcreteBuilder
- The Director class can manage the construction of a set of Product objects



When do you use the Builder Pattern?

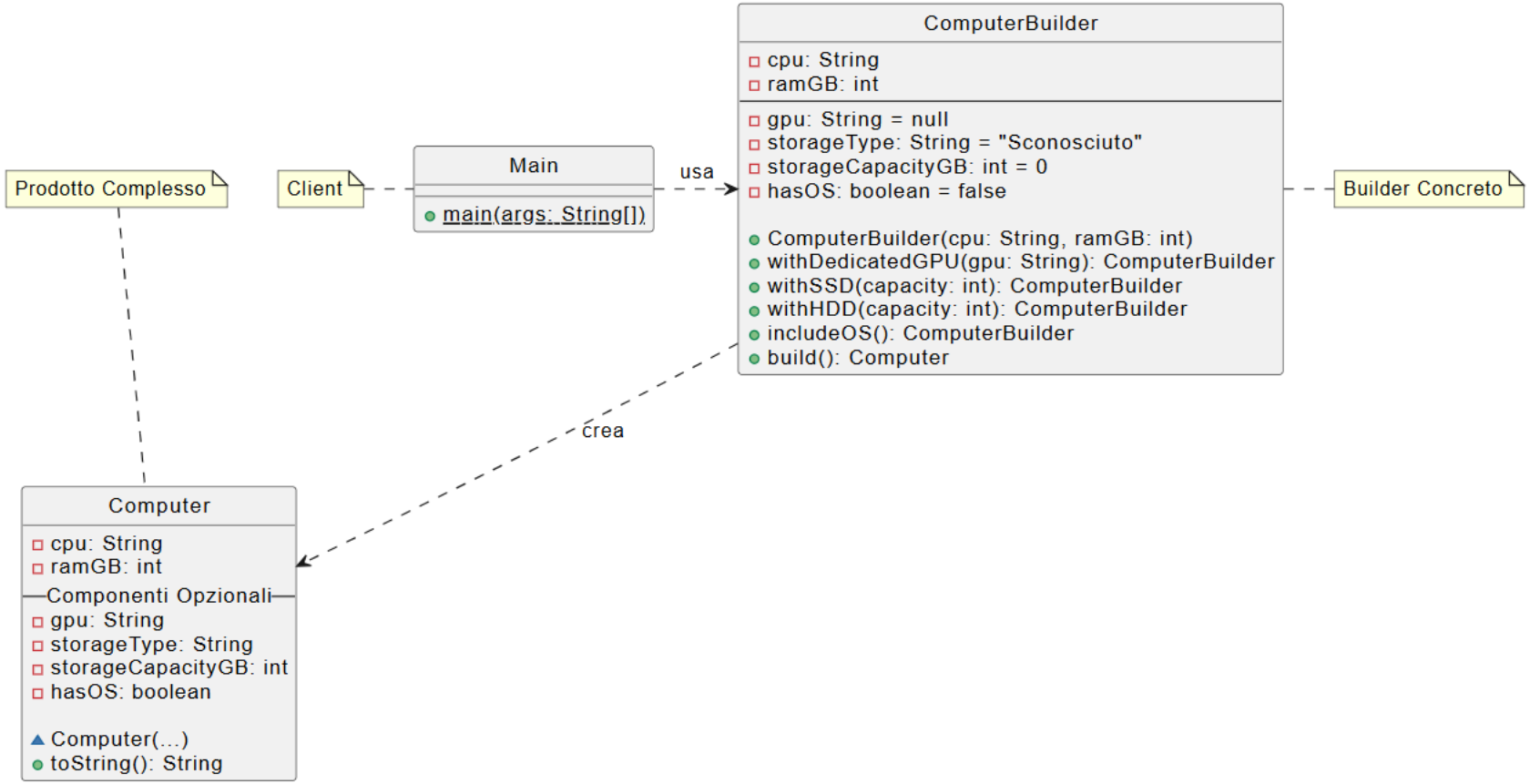
- The creation of a complex product must be independent of the particular parts that make up the product
  - In particular, the creation process should not know about the assembly process (how the parts are put together to make up the product)
- The creation process must allow different representations for the object that is constructed.
- Examples:
  - A house with one floor, 3 rooms, 2 hallways, 1 garage and three doors.
  - A skyscraper with 50 floors, 15 offices and 5 hallways on each floor.

Simple Example : Building a Computer

- We want to build objects of a Computer class, in order to manage their advertising, sale, etc.
- A computer can have many parameters, and you may need to add more parameters in the future
- We separate the Computer object from a ComputerBuilder object that only deals with managing all the possible variants of the construction of a Computer object in a flexible way

  - This simple example shows the use of a ConcreteBuilder class but not the use of the Director class

Class Diagram



Esempio: class Computer

```
public class Computer {
// Componenti obbligatori
private final String cpu;
private final int ramGB;


// Componenti opzionali
private final String gpu;
private final String storageType; // SSD o HDD
private final int storageCapacityGB;
private final boolean hasOS; // Sistema Operativo
```

```
/**
* Costruttore accessibile solo dal ComputerBuilder
* (presumendo che si trovino nello stesso package).
*/
Computer(String cpu, int ramGB, String gpu, String storageType, int
storageCapacityGB, boolean hasOS) {
this.cpu = cpu;
this.ramGB = ramGB;
this.gpu = gpu;
this.storageType = storageType;
this.storageCapacityGB = storageCapacityGB;
this.hasOS = hasOS;
}


}
```

Example: class ComputerBuilder

```
public class ComputerBuilder {
  // Parametri obbligatori (final, inizializzati nel costruttore del Builder)
  private final String cpu;
  private final int ramGB;

  // Parametri opzionali (con valori di default)
  private String gpu = null;
  private String storageType = "Sconosciuto";
  private int storageCapacityGB = 0;
  private boolean hasOS = false;

  // Costruttore per i parametri OBBLIGATORI del Computer.
public ComputerBuilder(String cpu, int ramGB) {
    this.cpu = cpu;
    this.ramGB = ramGB;
  }

  // Metodi setter opzionali: restituiscono il Builder stesso
  public ComputerBuilder withDedicatedGPU(String gpu) {
    this.gpu = gpu;
    return this;
  }

  public ComputerBuilder withSSD(int capacity) {
    this.storageType = "SSD";
    this.storageCapacityGB = capacity;
    return this;
  }

...
    // Metodo BUILD finale: crea e restituisce l'istanza finale del Computer.

public Computer build() {
    // Logica di validazione
    if (ramGB < 4) {
      throw new IllegalStateException("RAM insufficiente. Minimo richiesto: 4GB.");
    }
     // Chiama il costruttore package-private del Prodotto
    return new Computer(cpu, ramGB, gpu, storageType, storageCapacityGB, hasOS);
  }
}
```

Example : Main

```
public class Main {
public static void main(String[] args) {
// 1. Costruzione di un PC "Gaming" completo
System.out.println("--- Ordine 1: PC Gaming ---");
// le chiamate . sono innestate perché i vari metodi with di ComputerBuilder restituiscono lo stesso ComputerBuilder
Computer gamingPC = new ComputerBuilder("AMD Ryzen 7", 32).withDedicatedGPU("AMD Radeon RX 7800").withSSD(2000).includeOS().build();
System.out.println(gamingPC);


System.out.println("\n--- Ordine 2: PC Ufficio Base ---");
// Nessuna GPU dedicata, no OS
Computer officePC = new ComputerBuilder("Intel Core i3", 8).withHDD(500).build();
System.out.println(officePC);

System.out.println("\n--- Ordine 3: PC obsoleto ---");
Computer obsoletePC = new ComputerBuilder("Intel Core i3", 2)
.withHDD(250)
.build();
System.out.println(obsoletePC);

}
}
```

Example : Output

--- Ordine 1: PC Gaming ---

Configurazione Computer:

- CPU: AMD Ryzen 7
- RAM: 32GB
- GPU: AMD Radeon RX 7800
- Storage: 2000GB SSD
- OS Installato: true

--- Ordine 2: PC Ufficio Base ---

Configurazione Computer:

- CPU: Intel Core i3
- RAM: 8GB
- GPU: Integrata
- Storage: 500GB HDD
- OS Installato: false

--- Ordine 3: PC obsoleto ---

Exception in thread "main"
java.lang.IllegalStateException: RAM insufficiente. Minimo richiesto: 4GB.

at ComputerBuilder.build(ComputerBuilder.java:52)

at Main.main(Main.java:24)

The third PC has not been built: the logic related to obsolete computers is not in the Computer class but in the Builder class

Builder in Java libraries

- The Builder pattern is widely used in standard Java library classes:
  - java.lang.StringBuilder
    - Used to build strings
  - java.util.Locale.Builder
    - Used to build complex locale objects by defining language, country, scripts, and variants
  - java.time.format.DateTimeFormatterBuilder
    - Builder to create date-time formatters

Structural Design Patterns

Structural Design Patterns

- The patterns in this category are dedicated to the composition of classes and objects to create larger structures.
- You can create classes that inherit from multiple classes to allow you to use properties from multiple independent superclasses.
- For example, they allow independent libraries to work together.

Structural Design Patterns

Design Patterns	Description
Adapter	Converts the interface of one class to another allowing two classes to work together even if they have different interfaces.
Bridge	Decouple an abstraction from its implementation so that they can vary independently.
Composite	Compose objects in tree structures to implement recursive compositions
Decorator	It adds new responsibilities to an object dynamically, it is an alternative to subclasses to extend functionality

Structural Design Patterns

Design Patterns	Description
Facade	Provides a unified interface to a subsystem to make them easier to use
Proxy	Provides a surrogate for an object to control access
Flyweight	Use sharing to efficiently support a large number of fine-grained objects

Adapter

Design Pattern : Adapter

Name: Adapter

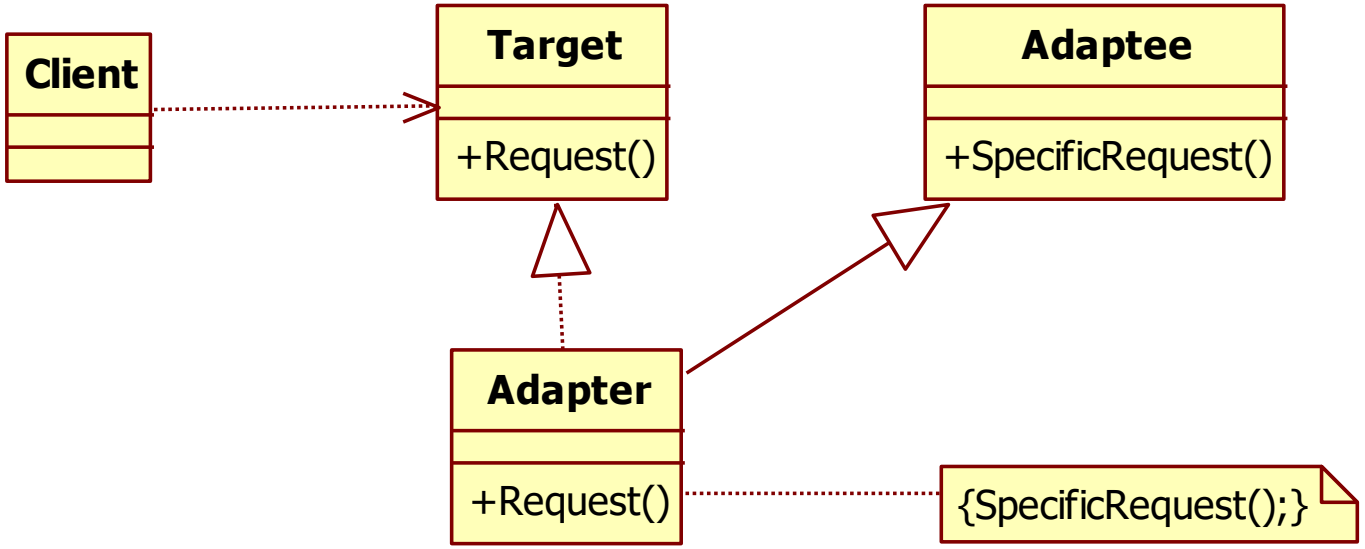
Synonym : Wrapper

Purpose : Converts the interface of one class into another interface, expected by a client (thus allowing the cooperation of classes that would otherwise have incompatible interfaces)

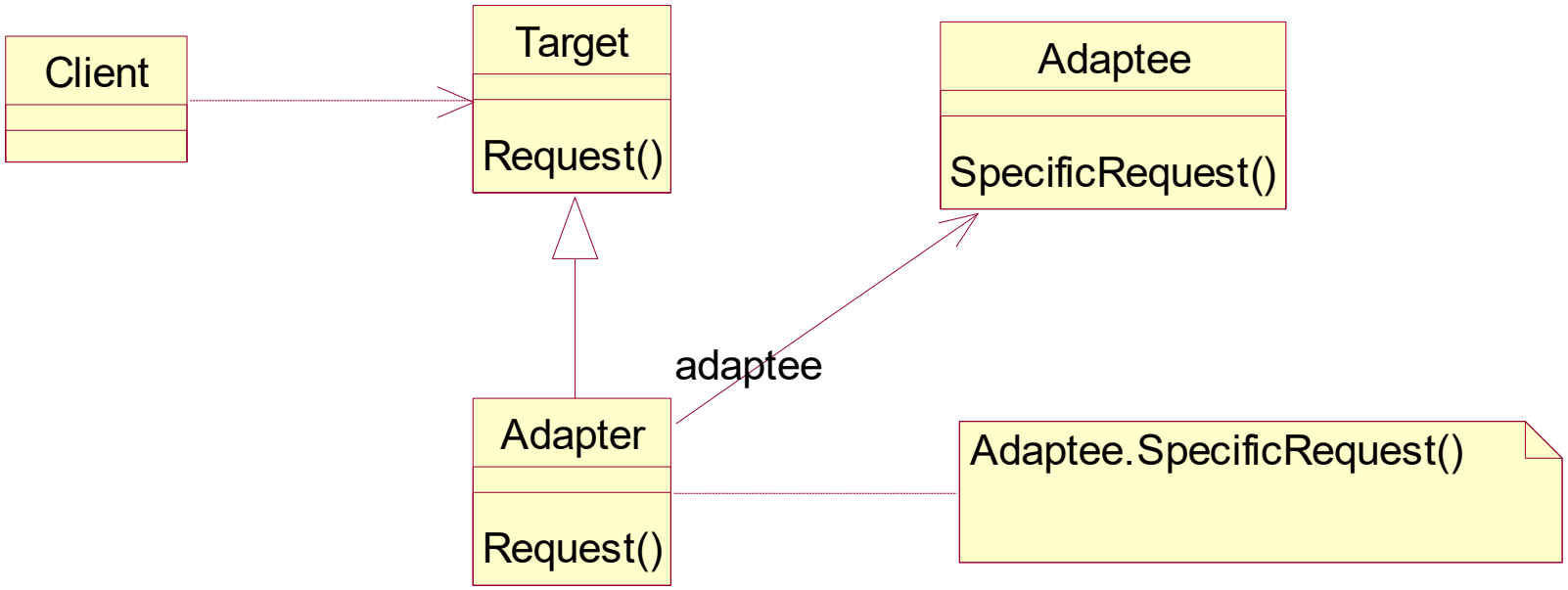
Motivation : When you want to use an already existing and tested class but it does not match the interface you need

Adapter

- **Participants :**
  - *Client* : the class that wants to use the service to be adapted
  - *Target* : the interface we would like from the service to use
  - *Adaptee* : the existing class that provides the service
  - *Adapter* : The class that adapts the interface provided by adaptee to the Target interface



Adapter : Alternative solution



Adapter

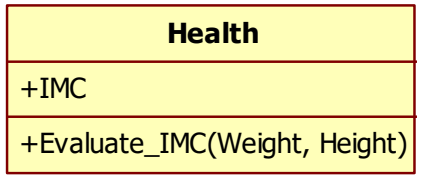
- **Uses**:

  Adapter can be used :

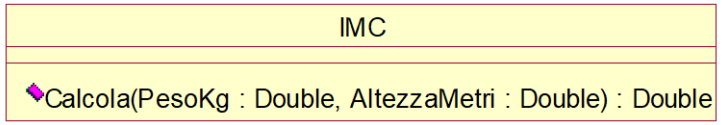
  1. If you want to use an existing class (Adaptee) whose interface is incompatible
  2. If you want to create a (reusable) class that will have to collaborate with classes that are not predictable at the time of its creation
  3. If you want to reuse a set of existing subclasses, but it is inconvenient to adapt the interface by creating a subclass for each: better to create an adapter for the parent class

Example

In the creation of an object-oriented software for the US market, we want to create a Health class (Client class) that provides, among other things, the body mass index (IMC) of a person, according to the interface shown below:



We do not know the algorithm that allows to calculate this index, thus we decided to reuse an IMC class (Adaptee class), taken from a software made for the Italian market that has the following interface:



We know that 1 pound is equal to 0.453 kilograms and 1 inch is equal to 0.0254 meters

We want to Design the class diagram at the level of detail that is able to solve the proposed problem by freeing the programmer of the Health class from the knowledge of the interface and implementation of the IMC class, using the Adapter design pattern.

Solution

```
public class Main {
   public static void main(String[] args) {
      Health health = new Health();
      health.evaluateIMC(165,70);
      System.out.println(health.IMC);
   }
}

//Classe che dobbiamo realizzare (inglese)

public class Health {
   public double IMC;
   public double evaluateIMC(double weight, double height) {
      Target adapter = new Adapter();
      IMC = adapter.evaluate(weight, height);
      return IMC;
   }
}

//interfaccia che richiediamo

public interface Target {
   public double evaluate(double weight, double height);
}
```

```
//Classe che va ad adattare la soluzione esistente

public class Adapter implements Target {
   public Adaptee adaptee = new Adaptee();
   public double evaluate(double weight, double height) {
      return adaptee.calcolaIMC(weight/0.453,height/0.0254);
   }
}


public class Adaptee {
   public double calcolaIMC(double pesoKg, double altezzaM){
      IMC imc = new IMC();
      return imc.calcolaIMC(pesoKg,altezzaM);
   }
}


public class IMC {
   public double calcolaIMC(double pesoKg, double altezzaM){
      return pesoKg/(altezzaM*altezzaM);
   }
}
```

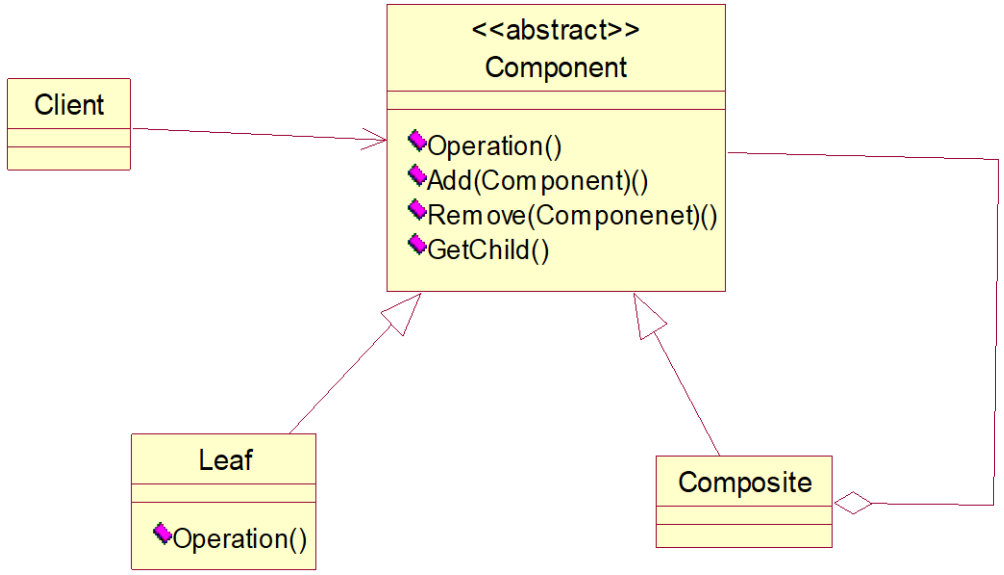
Typical Uses of Adapter

- The Adapter class intervenes continuously when dealing with services or microservices:
  - The design of our software makes us design an interface
  - An existing microservice provides the service we need, but with its own interface
  - We implement an Adapter class that interacts with the class that provides the service, converts the parameters provided by the service (Adaptee) into the ones we are interested in, possibly reorganizing and processing them, and provides them to the Target class
    - The target class could also be an Interface, since it only requests a service
  - Why don't we change the service (Adaptee) directly?
    - Because we didn't do it and we only have the interface or because it is already used by others

Composite

Design Pattern Composite

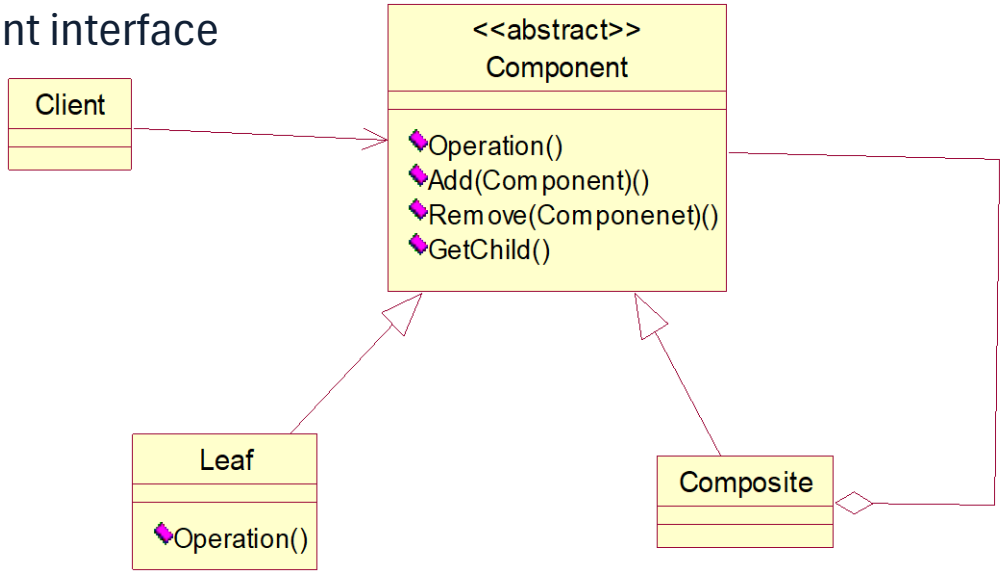
- Name : Composite
- Purpose: Compose objects into tree structures that can represent all-part hierarchies. You want clients to treat both composite objects and individual objects the same way



Design Pattern Composite

Partecipants

- **Component** declares the object interface and default behavior, declares an interface for accessing and managing its child components
- **Leaf** implements childless component objects and their behavior
- **Composite** defines the behavior of components with children, stores child components, implements child-related operations defined by the Component interface
- **Client** uses composition objects using the Component interface

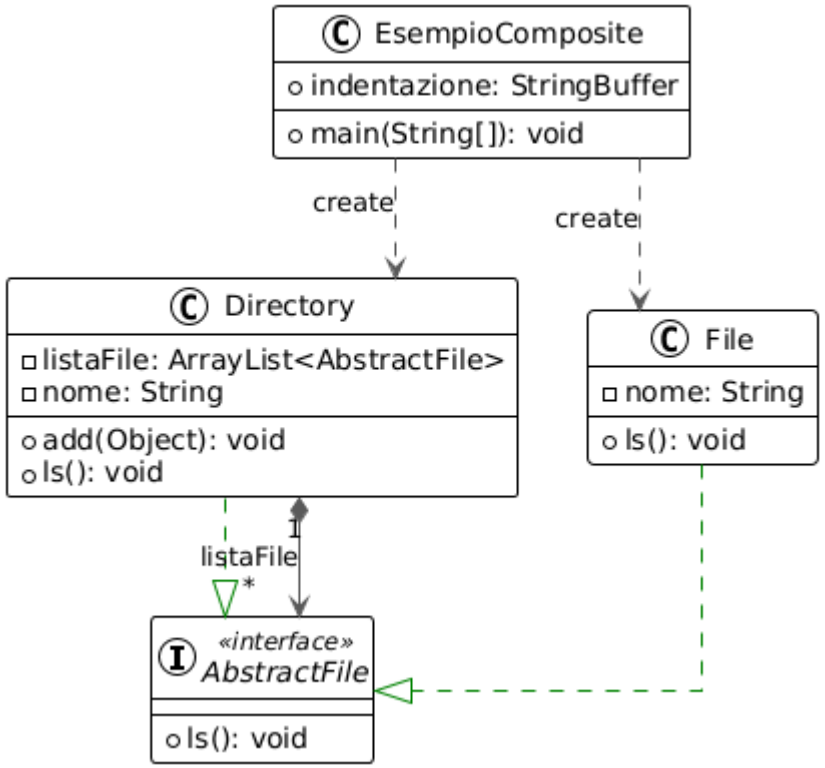


Example : File System

- We want to implement a directory and a file tree
  - With the **ls** function
  - Taking into account the *indentation*

- A **Directory** can be composed of **Files** and other directories, possibly empty

- **AbstractFile** represents items that can be included in Directories



Composite: Esempio (1/3)

```
interface AbstractFile
{
  public void ls();
}

class File implements AbstractFile
{
    public File(String n)
    {
        nome = n;
    }
    public void ls()
    {
        System.out.println(EsempioComposite.indentazione + nome);
    }
    private String nome;
}
```

Composite: Esempio (2/3)

```
class Directory implements AbstractFile {
    private String nome;
    private ArrayList listaFile = new ArrayList();

    public Directory(String name)
        { nome = name; }

    public void add(Object obj)
        { listaFile.add(obj); }

    public void ls()    {
        System.out.println(EsempioComposite.indentazione + nome);
        EsempioComposite.indentazione.append("   ");
        for (int i = 0; i < listaFile.size(); ++i)   {
            AbstractFile obj = (AbstractFile)listaFile.get(i); //Polimorfiscmo
            obj.ls();
        }
        EsempioComposite.indentazione.setLength(EsempioComposite.indentazione.length() - 3);
    }
}
```

Composite: Esempio (3/3)

```
public class EsempioComposite
{
    public static StringBuffer indentazione = new StringBuffer();

    public static void main(String[] args)
    {
        Directory one = new Directory("dir111"),
        two = new Directory("dir222"),
        thr = new Directory("dir333");
        File a = new File("a"), b = new File("b"),
        c = new File("c"), d = new File("d"), e = new File("e");
        one.add(a);
        one.add(two);
        one.add(b);
        two.add(c);
        two.add(d);
        two.add(thr);
        thr.add(e);
        one.ls();
    }
}
```

Output

```
dir111
  a
  dir222
    c
    d
    dir333
      e
  b
```

Consequences

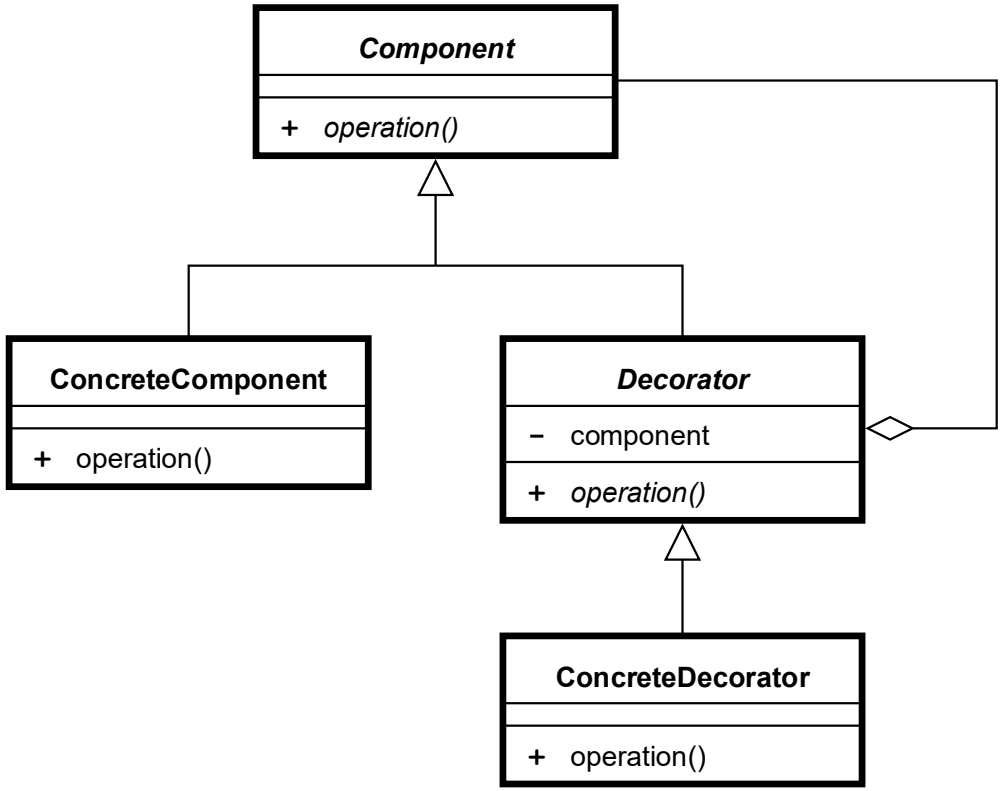
- Composite :
  - Allows to define class hierarchies consisting of primitive and composite objects.
    - Primitive objects can be composed to form more complex objects, which can be composed recursively.
    - At all points where the client expects to use a primitive object, a composite object can be used indifferently.
  - Simplifies the client.
    - Clients can use composite structures and individual objects uniformly.
    - Clients usually don't know (and shouldn't even care) whether they're working with a leaf or a composite component.
  - Is flexible
    - Makes it easier to add new types of components (new Leaf or Composite subclasses can be automatically used in existing structures and work with client code)
  - Is often used to instantiate the hierarchy of widgets of a GUI

Decorator

Pattern Decorator

- The pattern decorator allows to create complex objects by adding functionality to simpler objects.
- The complex object includes the simple object and "decorates" it by adding functionality
  - The original object is passed to the decorator's constructor as a parameter and the "decorated" object is returned
  - Multiple decorators can also be chained to each other, thus incrementally adding functionality to the complex class (which is represented by the last link in the chain).
    - The concatenation of decorators can take place according to an arbitrary composition: the number of possible behaviors of the composite object therefore varies with combinatorial law with respect to the number of decorators available.
- This pattern is an alternative to the use of single or multiple inheritance. With inheritance, in fact, the addition of features occurs statically according to the links defined in the class hierarchy and it is not possible to obtain at run-time an arbitrary combination of features, nor their addition/removal.

Schema concettuale



- Component is abstract/interface
- ConcreteComponent is the basic implementation of component
- Decorator is an abstract class that includes a Component and inherit operation
- ConcreteDecorator is a class that improves Component because it can take on all its characteristics (it has a component attribute) and can modify its methods in override

Pizza Example: Interface, Decorator, and Base Class

```
//component
public interface Pizza {String getDescription();
double getCost();
}

//Decorator
public abstract class PizzaDecorator implements Pizza {
protected Pizza pizza;

public PizzaDecorator(Pizza pizza) {
this.pizza = pizza;}

@Override
public String getDescription() {
return pizza.getDescription(); }

@Override
public double getCost() {
return pizza.getCost(); }
}

//ConcreteComponent
public class OlioEPomodoro implements Pizza {

@Override
public String getDescription() {
return "Pizza Olio e Pomodoro";}

@Override
public double getCost() {
return 3.00;}
}
```

Example : decorators

```
// Concrete Decorators
public class Mozzarella extends PizzaDecorator {
public Mozzarella(Pizza pizza) {
super(pizza); } //Mozzarella è una Pizza

@Override
public String getDescription() {
return pizza.getDescription() + ", Mozzarella";}

@Override
public double getCost() {
return pizza.getCost() + 1.50;}
}
```

```
public class Prosciutto extends PizzaDecorator {
public Prosciutto(Pizza pizza) {
super(pizza); } // Prosciutto è una Pizza

@Override
public String getDescription() {
return pizza.getDescription() + ", Prosciutto"; }

@Override
public double getCost() {
return pizza.getCost() + 1.00; }
}
```

Pizza example : Main

```
public class Main {
public static void main(String[] args) {
// Ordiniamo una pizza base
Pizza myPizza = new OlioEPomodoro();
System.out.println("Pizza ordinata: " + myPizza.getDescription());
System.out.println("Costo: €" + myPizza.getCost());
System.out.println("--------------------");

// Aggiungiamo formaggio alla pizza
myPizza = new Mozzarella(myPizza);
System.out.println("Pizza ordinata: " + myPizza.getDescription());
System.out.println("Costo: €" + myPizza.getCost());
System.out.println("--------------------");

// Pizza doppia Mozzarella
Pizza mia = new OlioEPomodoro();
mia = new Mozzarella(mia);
mia = new Mozzarella(mia);
System.out.println("\n*** La mia pizza ***");
System.out.println("Descrizione: " + mia.getDescription());
System.out.println("Costo totale: €" + mia.getCost());

//prosciutto e funghi
Pizza tua= new OlioEPomodoro();
tua = new Prosciutto(tua);
tua = new Fungo(tua);
System.out.println("\n*** La tua pizza ***");
System.out.println("Descrizione: " + tua.getDescription());
System.out.println("Costo totale: €" + tua.getCost());
}
}
```

Decorators in Java

- The Decorator pattern is often used within the Java libraries
- Examples:
  - Reader()
  - BufferedReader(Reader in)
  - InputStream()
  - InputStreamReader(InputStream in)
  - BufferedInputStream(InputStream in)

```
BufferedReader in = new BufferedReader(new
  InputStreamReader(System.in));
```

Decorator : Advantages and disadvantages

- Advantages :
  - By combining decorators we can get an unlimited number of combinations of builder calls
    - We don't need to declare classes «prosciutto e funghi», «margherita», «margherita prosciutto e funghi», «margherita doppia mozzarella»

- Disadvantage :
  - The construction of an object implies the need to instantiate more than one object and to call more than one constructor: the pattern is inefficient from a computational point of view (and, consequently, also from an energy point of view)

Behavioral Design Patterns

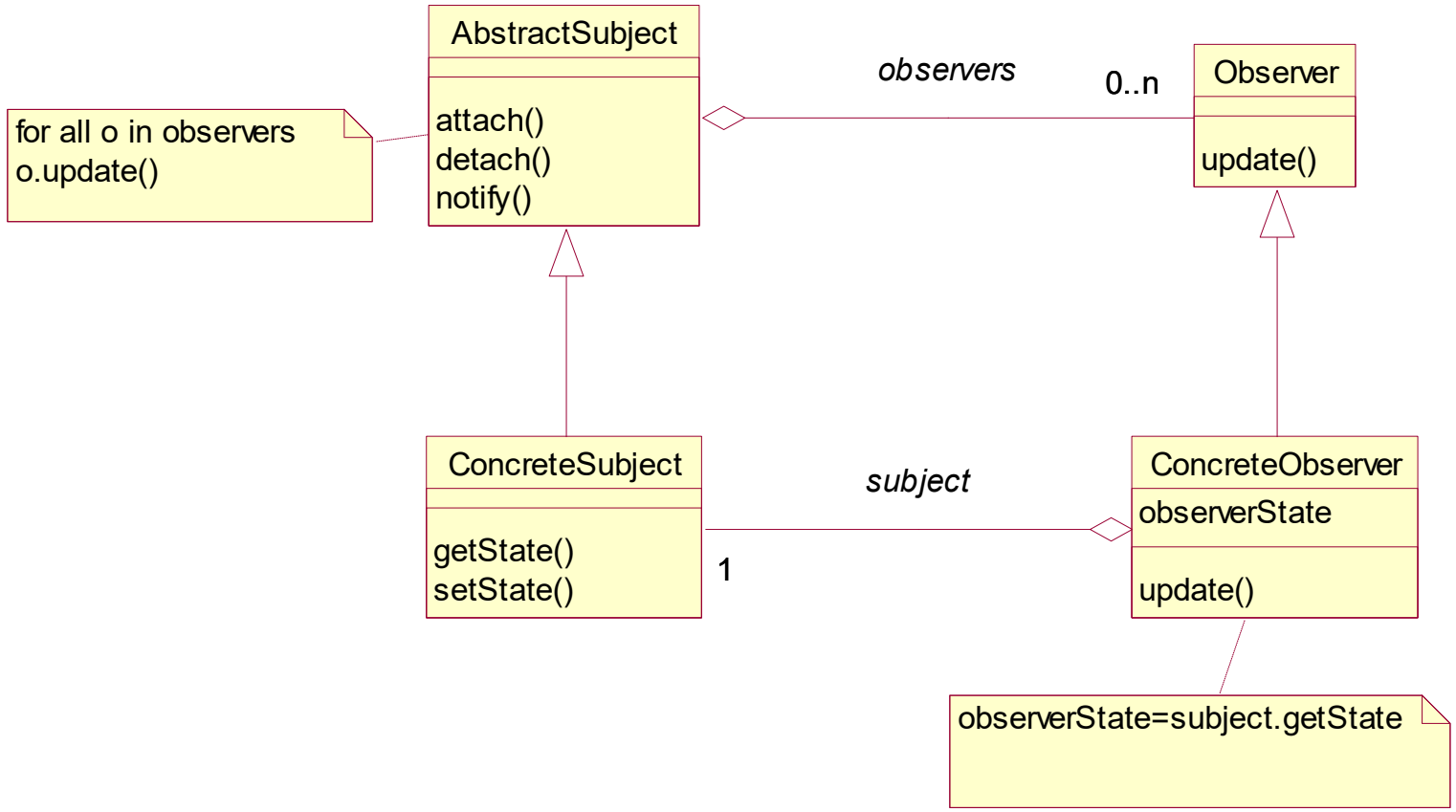
Design Patterns	Description
Observer	Defines a 1:N dependency between objects so that if one changes state, the others are automatically updated
State	It allows an object to change its behavior depending on its internal state
Strategy	It defines a family of algorithms, encapsulates each one and makes them interchangeable so that they can change independently of the users
Template method	Defines the skeleton of an algorithm in an operation by letting subclasses define some steps
Visitor	Represents an operation on the elements of an object's structure. New operations can be defined without changing element classes

Observer

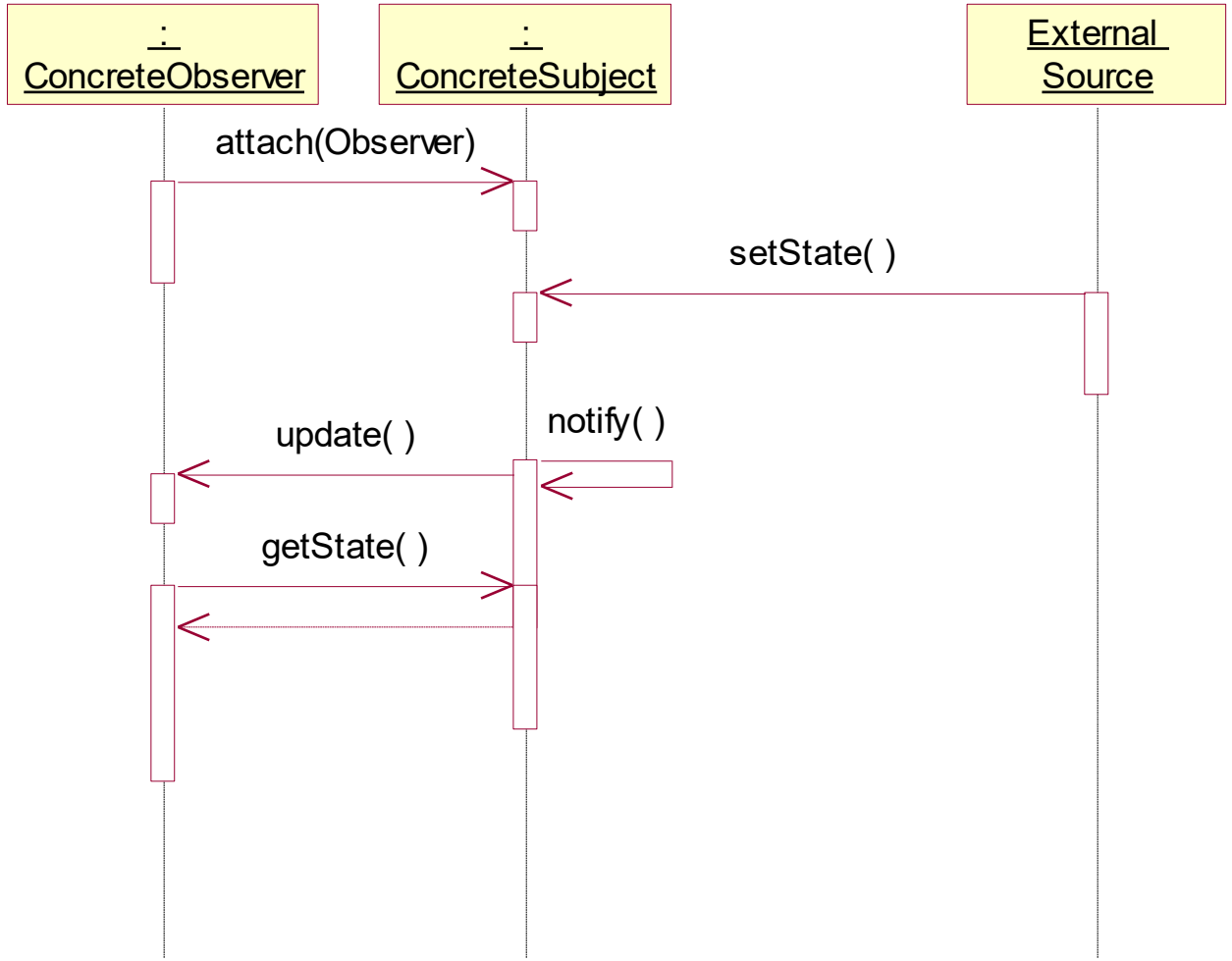
Observer

- Context:
  - One or more objects are interested in observing the changes in the state of a subject
- Problem:
  - The subject may have different observers, even of different types
  - It should be possible to add new observers while the application is running
- Solution:
  - include in the subject operations that allow the observer to declare his interest in a change of state

Observer


AbstractSubject
attach()
detach()
notify()
for all o in observers
o.update()
observers
0..n
Observer
update()
ConcreteSubject
getState()
setState()
subject
1
ConcreteObserver
observerState
update()
observerState=subject.getState

Observer: behaviour



Esempio 1/2

```
//osservatore (interfaccia) – invariante – potrebbe essere una interfaccia di libreria
public interface Osservatore {
   public void aggiorna();
}



//osservatore (concreto)

//e' legato ad un soggetto del quale vuole conoscere le modifiche
public class OsservatoreConcreto implements Osservatore {
   private int stato; //stato dell'osservatore
   private Soggetto osservato; // soggetto osservato

   public OsservatoreConcreto(Soggetto soggetto) {
     this.osservato=soggetto;      //soggetto osservato
   }


 @Override
   public void aggiorna() {
     stato = osservato.getStato();
     System.out.println(«Notificato; Nuovo Stato:");
     System.out.println(stato);
   }
}
```

```
//soggetto (astratto) – invariante – potrebbe essere una classe di libreria

public abstract class Soggetto {
   protected int stato; //lo stato del soggetto originale
   protected ArrayList<Osservatore> osservatori;

   public void registraOsservatore (Osservatore Osvtr) {
     osservatori.add(Osvtr);
   }


// informa osservatori del cambiamento di stato

   protected void update() {
     for (int i=0; i<osservatori.size(); i++) {
       Osservatore Osvtr = osservatori.get(i);
       Osvtr.aggiorna();
     }
   }
   public abstract void setStato(int s);
   public abstract int getStato();
}
```

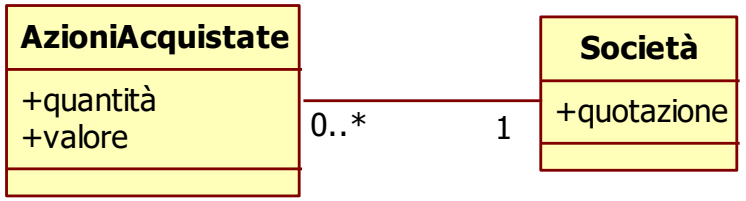
```
//soggetto (concreto)

public class SoggettoConcreto extends Soggetto {
   public SoggettoConcreto() {
     osservatori=new ArrayList<Osservatore>();
     stato=0;
   }
   @Override
   public void setStato(int s) {
     stato = s;
     update(); //rispetto ad una normale classe c'è questa chiamata extra
   }
   @Override
   public int getStato() {
     return stato;
   }
}
```

Esempio 2/2

```
public class Main {

  public static void main(String[] args) {
    Soggetto soggetto = new SoggettoConcreto();
    Osservatore o1 = new OsservatoreConcreto (soggetto);
    soggetto.registraOsservatore(o1);

    Osservatore o2 = new OsservatoreConcreto (soggetto);
    soggetto.registraOsservatore(o2);

    soggetto.setStato(20);
  }
}
```

Problema

Si vuole gestire un attributo valore nella classe AzioniAcquistate che misuri il valore istantaneo di un insieme di azioni acquistate.

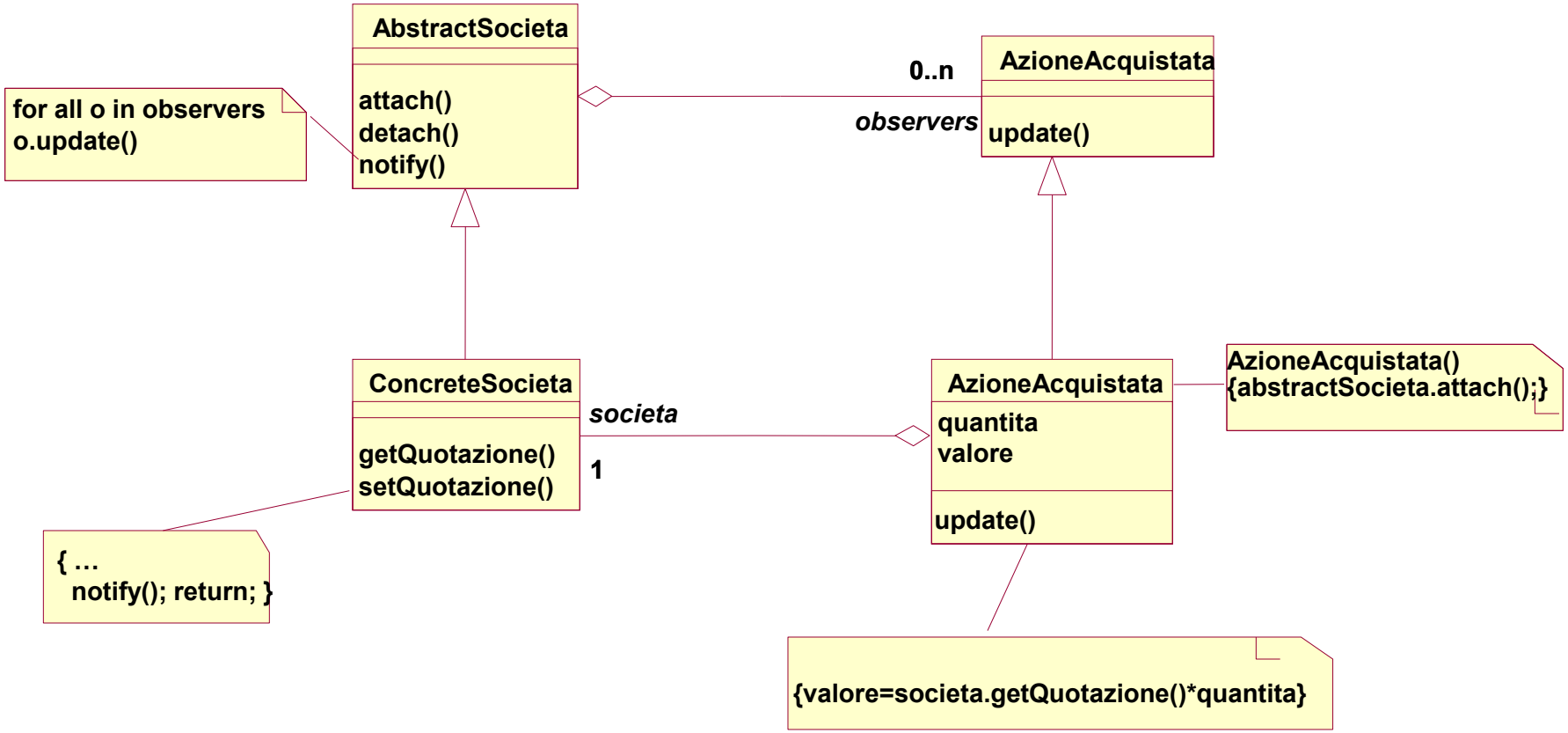


Esso è calcolato come:

AzioniAcquistate.valore = Società.quotazione * AzioniAcquistate.quantità

Allorché il sistema aggiorna il valore della quotazione delle azioni di una società (ovvero l'attributo *Società.quotazione*), si vuole che gli attributi *valore* delle istanze delle classi *AzioniAcquistate* siano automaticamente aggiornati.

Soluzione con observer



When to use the Observer

- When changes on a subject need to be "broadcasted" to many objects

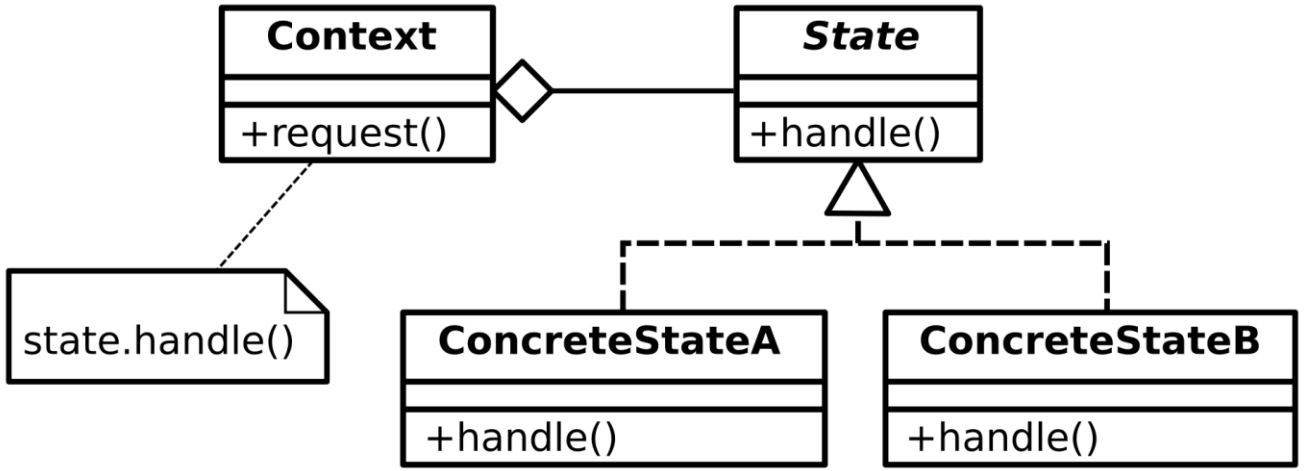
- When application-independent objects (subjects) need to invoke application-specific objects to notify a change in their internal state (subjects don't need to know anything about their observers or how they work)
  - For example when a sensor or a service provides am update for a datum and the user interface has to be notified about this update

- The Observer is the heart of the MVC pattern, in which the objects of the user interface (View - GUI) can be direct observers of the objects of the model (Model)
  - In this case, Model objects have notify functions, while GUI objects can have update functions

State

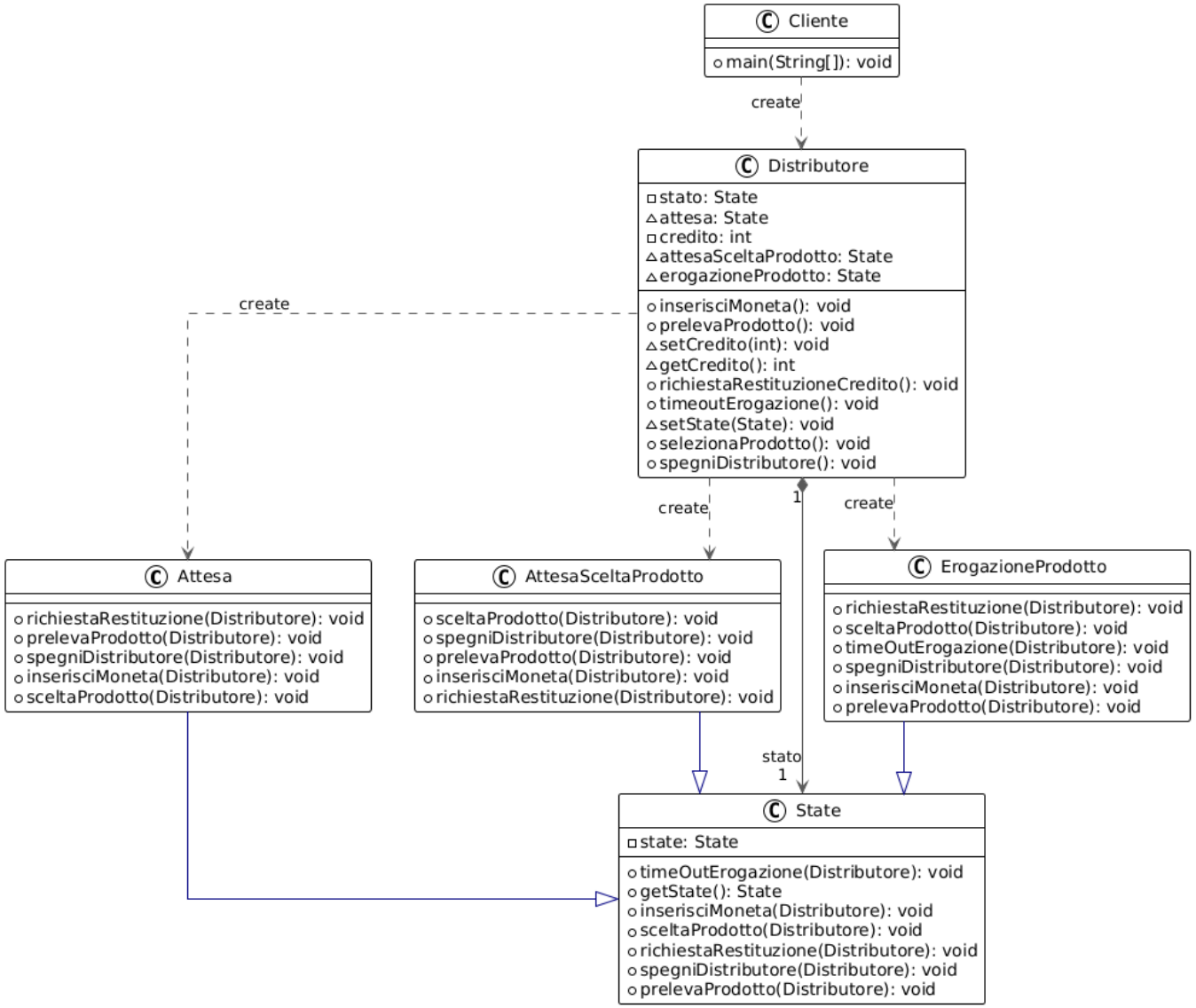
Design Pattern State

- A solution for general and flexible modeling of state machines
- The State class contains empty implementations of all state transition methods (triggers – handler) and hosts attributes that represent the state of the machine. The polymorphism mechanism allows Context to have an attribute of the State class and interact with any possible state



Pattern State

- A more effective solution in object-oriented systems is to use the pattern state
- The statechart is translated as:
  - A general class that declares the methods corresponding to all actions but implements them without doing anything
  - A series of derived classes, corresponding to the states of the diagram, which implement only the methods corresponding to the events that can be triggered on that state (by overriding)



The code of this solution is on Teams

Example of Pattern State

Distributore Class (container)

```
public class Distributore {
   private State stato;
   static State attesaSceltaProdotto = new AttesaSceltaProdotto();
   static State attesa = new Attesa();
   static State erogazioneProdotto = new ErogazioneProdotto();
   private int credito=0;

   public Distributore() {setState(attesa); }
   void setState (State newStato) {      this.stato=newStato;    }

   void setCredito(int credito) {this.credito=credito;}
  int getCredito() { return credito; }
   public void inserisciMoneta() {      stato.inserisciMoneta(this);    }
   public void selezionaProdotto() {      stato.sceltaProdotto(this);    }
   public void richiestaRestituzioneCredito()
{      stato.richiestaRestituzione(this);    }
   public void spegniDistributore() {      stato.spegniDistributore(this);    }
   public void prelevaProdotto() {      stato.prelevaProdotto(this);    }
   public void timeoutErogazione() { stato.timeOutErogazione(this); }
}
```

Generic Class for States

```
public class State {
   private State state;

   public State getState() {
     return state;
   }

   public  void inserisciMoneta(Distributore distributore) {}

   public  void richiestaRestituzione(Distributore distributore) {}

  public  void sceltaProdotto(Distributore distributore) {}

   public  void spegniDistributore(Distributore distributore) {}

   public  void prelevaProdotto(Distributore distributore) {}
   public void timeOutErogazione(Distributore distributore){}
}
```

Fragments of code

State Attesa Class

```
• public class Attesa extends State {

    @Override
    public void inserisciMoneta(Distributore distributore) {
      distributore.setCredito(distributore.getCredito()+1);
      System.out.println("Credito attuale : "+distributore.getCredito());
      if (distributore.getCredito()>=2){
        distributore.setState(Distributore.attesaSceltaProdotto);
      }
    }

    @Override
    public void richiestaRestituzione(Distributore distributore) {
      System.out.println("Ritira il tuo credito : "+distributore.getCredito());
      distributore.setCredito(0);
    }

    @Override
    public void sceltaProdotto(Distributore distributore) {
      System.out.println("Non hai credito sufficiente");
    }

    @Override
    public void spegniDistributore(Distributore distributore) {
      System.out.println("Ritira il tuo credito : "+distributore.getCredito());
      distributore.setCredito(0);
      System.out.println("Spengo il distributore");
      System.exit(0);
    }

    @Override
    public void prelevaProdotto(Distributore distributore) {
      System.out.println("Il prodotto non è pronto");
    }
  }
```

Example Main (for testing)

```
public class Cliente {

  public static void main(String[] args) {
    Distributore dis = new Distributore();

    System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
//selezioniamo un prodotto senza prima inserire una moneta
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Scelgo un prodotto. ");
dis.selezionaProdotto();//selezioniamo un prodotto
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
//seleziona il prodotto
    System.out.print("Prelevo il prodotto. "); dis.prelevaProdotto();
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
    System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();
    System.out.print("Richiedo la restituzione del credito. ");
dis.richiestaRestituzioneCredito();
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
    System.out.print("Prelevo il prodotto. "); dis.prelevaProdotto();
    System.out.print("Spengo il distributore. "); dis.spegniDistributore();
    System.out.print("Inserisco un euro. "); dis.inserisciMoneta();
  }
```

Polymorphism is used to adapt the behavior with respect to different states

Modern Patterns

Not included in the original GoF set of patterns

Dependency Injection

Dependency Injection

- It occurs when the resources on which a component depends are passed (injected) from outside rather than instantiated by the component itself during its execution
- The component have to declare (for example through its constructor) the type (possibly generic) and the quantity of all the resources on which it depends
- References to the required resources will then be passed (injected) e.g. through the constructor

Simple example

```
public class MyClass {
    private Logger logger;

    public MyClass(Logger logger) {
        this.logger = logger;
        // write an info log message
        logger.info("This is a log message.")
    }
}
```

- MyClass can write into a log
- The log object on which it writes is given by the object of type Logger passed as a parameter
- https://www.vogella.com/tutorials/DependencyInjection/article.html

Pattern consequences

- It makes the class more generic, as it separates its behavior from the specific object being passed
  - It introduces a dependency between an object and a class
- It may not be very optimal from a coupling point of view
- It is also called **inversion-of-control** because it requires to create first the dependent object and then the one that will use it, going in the opposite direction to, for example, the BCE model

Dependency Injection and Testing

- Dependency Injection is fundamental for isolation and integration testing
  - It allows to perform a test on an object having entered as input a fictitious object on which it acts, replacing the concrete object that may not be available or not yet tested

- Example:
  - you want to test a class that calls a service, but the service is not yet available.
  - If the *dependency injection* pattern is followed, then the class under test will receive via the constructor a reference to the service object
  - When we are testing the class, we can create a fictitious service object and pass it to the class
  - In this way, the class can be executed even without the original service
- Counterexample:
  - If the class does not follow this pattern, the reference to the service is hardcoded in the class under test or is instantiated in it and cannot be modified at testing time

Conclusions

Summary

- Structural Patterns
  - Focus: How objects are composed to form larger structures
  - Problems solved:
    - Realize new functionality from old functionality,
    - Provide flexibility and extensibility
- Behavioral Patterns
  - Focus: Algorithms and the assignment of responsibilities to objects
  - Problem solved:
    - Too tight coupling to a particular algorithm
- Creational Patterns
  - Focus: Creation of complex objects
  - Problems solved:
    - Hide how complex objects are created and put together

Observations

- Patterns permit design at a more abstract level
  - Treat many class/object interactions as a unit
  - Often beneficial after initial design
  - Targets for class refactorings
- Variation-oriented design
  - Consider what design aspects are variable
  - Identify applicable pattern(s)
  - Vary patterns to evaluate tradeoffs
  - Repeat

Conclusion

- Design patterns
  - Provide solutions to common problems.
  - Lead to extensible models and code.
  - Can be used as is or as examples of interface inheritance and delegation.
  - Apply the same principles to structure and to behavior.

- Design patterns solve all your software engineering problems

Conclusions

- We could go on and on and present different patterns.
- However, at the end of the day we need to first define our problem before we come up with a solution.
  - **And since patterns are all about solutions to a problem, don't look at patterns until you have already defined the problem!**

(Design) Pattern References

- The Timeless Way of Building, Alexander; Oxford, 1979; ISBN 0-19-502402-8
- A Pattern Language, Alexander; Oxford, 1977; ISBN 0-19-501-919-9
- Design Patterns, Gamma, et al.; Addison-Wesley, 1995; ISBN 0-201-63361-2; CD version ISBN 0-201-63498-8
- Pattern-Oriented Software Architecture, Buschmann, et al.; Wiley, 1996; ISBN 0-471-95869-7
- Analysis Patterns, Fowler; Addison-Wesley, 1996; ISBN 0-201-89542-0
- Smalltalk Best Practice Patterns, Beck; Prentice Hall, 1997; ISBN 0-13-476904-X
- The Design Patterns Smalltalk Companion, Alpert, et al.; Addison-Wesley, 1998; ISBN 0-201-18462-1
- AntiPatterns, Brown, et al.; Wiley, 1998; ISBN 0-471-19713-0